PROTOCORD

DOCUMENTAÇÃO TÉCNICA CONSOLIDADA DO PROTOCORD

Brasil
2026

RESUMO

Este documento apresenta a documentação técnica consolidada do ProtoCord, composto por uma interface web e por um servidor em funções sem servidor. A interface é responsável pela experiência operacional do usuário, enquanto o servidor centraliza persistência, integrações, transcrição de áudio, inteligência artificial e proteção de credenciais. O conteúdo descreve arquitetura, fluxos de dados, modelo entidade-relacionamento, rotas, variáveis, escopos funcionais, instalação, execução, testes e recomendações de manutenção.

Palavras-chave: ProtoCord; suporte técnico; transcrição de áudio; inteligência artificial; documentação de software.

SUMÁRIO

1. Introdução

2. Visão geral da solução

3. Arquitetura da interface

4. Arquitetura do servidor

5. Fluxo operacional principal

6. Modelo entidade-relacionamento

7. Documentação da interface

8. Documentação do servidor

9. Instalação, execução e testes

10. Contrato de API

11. Recomendações técnicas

12. Referências internas

1. INTRODUÇÃO

O ProtoCord é uma solução de apoio a analistas de suporte. O sistema permite registrar protocolos, consultar histórico, acompanhar liberações, transcrever áudios de atendimento e utilizar um assistente com inteligência artificial baseado em contexto operacional.

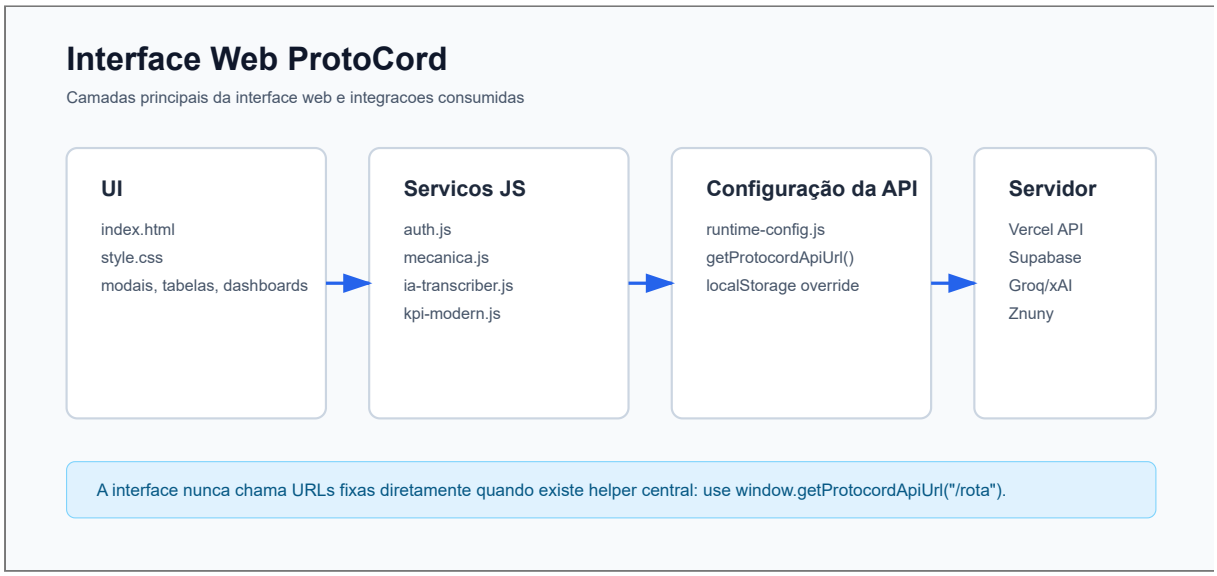
Esta documentação foi elaborada para orientar abertura do repositório, implantação local, manutenção técnica, apresentação do projeto e análise de sua estrutura interna.

2. VISÃO GERAL DA SOLUÇÃO

Projeto	Responsabilidade
modelo-discord	Interface web estática, executada no navegador.
modelo-discord-server	Servidor em Node.js, publicado em funções sem servidor da Vercel.

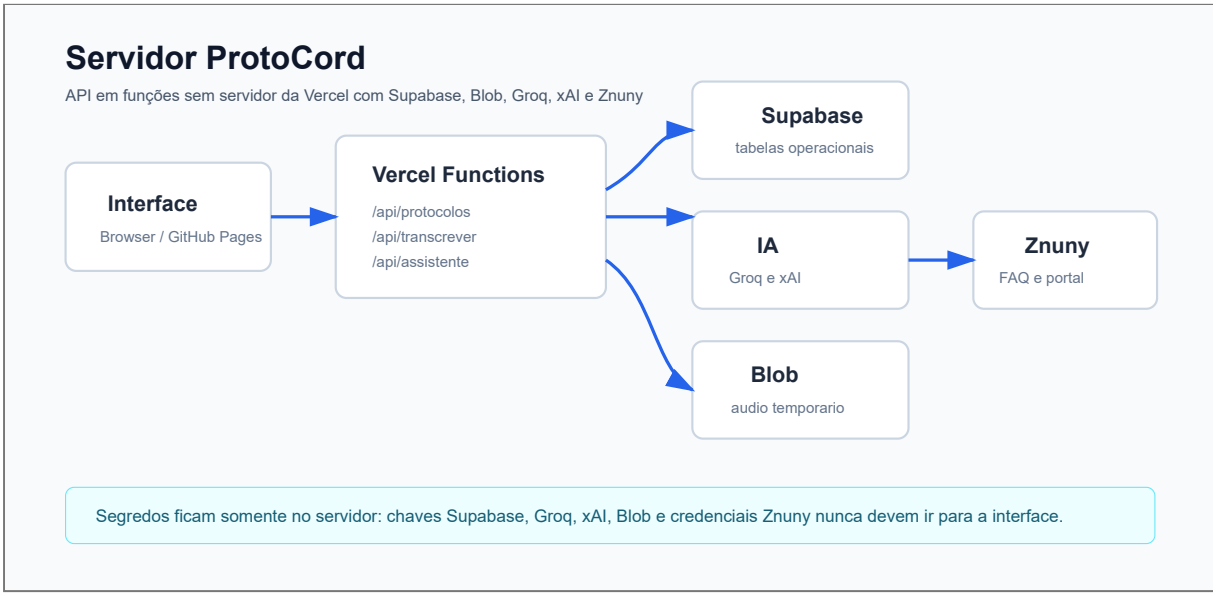
A interface comunica-se exclusivamente com o servidor por meio de rotas HTTP. O servidor integra-se ao Supabase, ao Vercel Blob, ao Groq, ao xAI e ao Znuny.

3. ARQUITETURA DA INTERFACE



A interface concentra componentes visuais, formulários, painéis, tabelas, gráficos e widgets. A configuração da API é resolvida em tempo de execução pelo arquivo `js/runtime-config.js`.

4. ARQUITETURA DO SERVIDOR

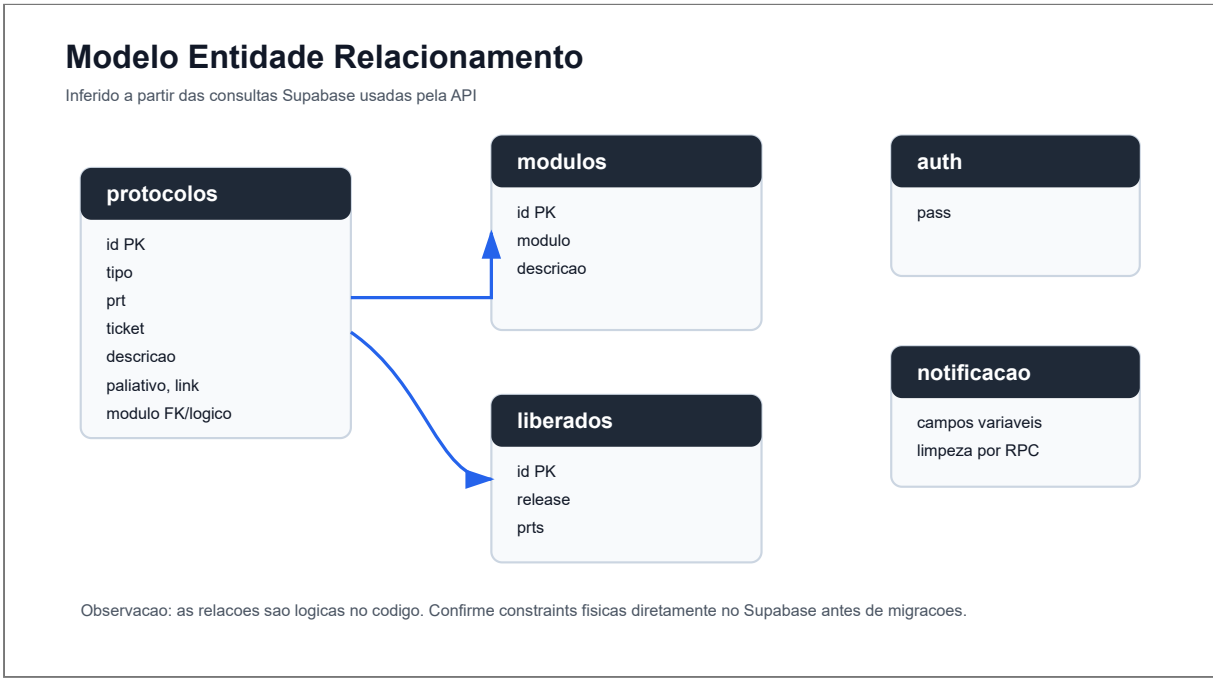


O servidor isola segredos e integrações externas. Todas as chaves de acesso devem permanecer em variáveis de ambiente no ambiente de execução.

5. FLUXO OPERACIONAL PRINCIPAL

1. O usuário abre a interface.
2. A interface resolve a URL do servidor por `js/runtime-config.js`.
3. A autenticação chama a rota `/api/autenticacao`.
4. Protocolos, módulos, liberações e notificações são carregados do servidor.
5. Ações de cadastro e exclusão são enviadas às rotas HTTP.
6. Áudios são enviados temporariamente ao Vercel Blob e processados por `/api/transcrever`.
7. O assistente utiliza Supabase, FAQ pública, portal Znuny e provedores de inteligência artificial para responder com base em contexto.

6. MODELO ENTIDADE-RELACIONAMENTO



Entidade	Finalidade
protocolos	Base de PRTs, tickets, descrições, paliativos, ligações de referência e módulos.
modulos	Catálogo de módulos para filtros, exibição e contexto.
liberados	Histórico de versões e PRTs liberados.
auth	Senha MD5 usada na autenticação atual.
notificacao	Alertas operacionais.

7. DOCUMENTAÇÃO DA INTERFACE

7.1 ESCOPO

A interface apresenta a experiência do usuário final. Ela valida formulários, exibe dados operacionais, renderiza painéis e aciona o servidor para qualquer operação que envolva dados, arquivos ou inteligência artificial.

7.2 ARQUIVOS PRINCIPAIS

Arquivo	Função

index.html	Documento principal da aplicação.
style.css	Estilos globais e componentes visuais.
js/runtime-config.js	Configuração da URL do servidor.
js/auth.js	Sessão e validação de senha.
js/mecanica.js	Fluxo principal de protocolos.
js/ia-transcriber.js	Gravação, envio e transcrição de áudio.
js/ia-assistant*.js	Assistente com inteligência artificial.
js/kpi-modern.js	Indicadores operacionais.
js/dashboard-liberacoes.js	Painel de liberações.

8. DOCUMENTAÇÃO DO SERVIDOR

8.1 ESCOPO

O servidor concentra regras de integração, persistência, transcrição, análise por inteligência artificial e respostas contextuais. Nenhum segredo deve ser exposto na interface.

8.2 ROTAS

Rota	Métodos	Função
/api/health	GET	Diagnóstico do ambiente.
/api/autenticacao	GET	Validação de senha MD5.
/api/protocolos	GET, POST, DELETE	Operações de protocolos.
/api/modulos	GET	Consulta de módulos.
/api/liberados	GET, POST	Consulta e registro de liberações.
/api/notificacao	GET	Consulta de notificações.
/api/limpar_notificacao	GET	Limpeza de notificações por função remota.
/api/blob-upload	POST	Preparação de envio temporário de áudio.
/api/excluir-audio	POST, DELETE	Remoção de arquivo temporário.
/api/transcricao-direta	POST	Transcrição direta de áudio.

/api/transcrever	GET, POST	Transcrição e análise estruturada.
/api/assistente	GET, POST	Assistente contextual.

9. INSTALAÇÃO, EXECUÇÃO E TESTES

```
cd modelo-discord
npm install
npx serve . -l 5500

cd ../modelo-discord-server
npm install
npx vercel dev

npm test
```

10. CONTRATO DE API

O servidor contém o arquivo `openapi.yaml`, compatível com ferramentas de documentação baseadas no padrão OpenAPI 3.0.

11. RECOMENDAÇÕES TÉCNICAS

- Centralizar CORS e respostas de erro em função auxiliar compartilhada.
- Substituir autenticação baseada em MD5 por sessão com usuário real e token assinado.
- Versionar o esquema do Supabase com migrações.
- Publicar uma interface visual da documentação da API a partir de `openapi.yaml`.
- Adicionar registros estruturados e correlação por requisição.
- Corrigir definitivamente a codificação dos arquivos históricos para UTF-8.

12. REFERÊNCIAS INTERNAS

- modelo-discord/README.md
- modelo-discord/docs/DOCUMENTACAO_TECNICA.md
- modelo-discord-server/README.md
- modelo-discord-server/docs/DOCUMENTACAO_TECNICA.md
- modelo-discord-server/openapi.yaml